EXTREME-ACCURACY VERIFICATION / ELI5 EDITION

KLB Seedchain GPU v0.5.0

Full SGP4 / SDP4 on an RTX 5070 Ti Laptop GPU

VALIDATED FOR ITS STATED COMPUTE ROLE

WHAT PASSED	WHAT THIS DOES NOT CLAIM
Full Vallado-style near-Earth and deep-space propagation; CPU/GPU agreement; independent implementation cross-check; pass-event identity; sustained laptop feasibility; container integrity.	Formal proof over every possible orbit, a full ITRF/EOP navigation stack, antenna and RF modeling, stale-element prediction accuracy, or operational safety certification.

Prepared 16 August 2026 | Windows 11 | CUDA 12.8 | driver 591.59
Package: klb_seedchain_gpu_v0.5.0_full_sgp4

BOTTOM LINE

Yes, the SGP4 approach works on this laptop

The package successfully stores compact orbital seeds and regenerates positions, velocities, and pass events on demand with the full SGP4/SDP4 branch set. The RTX 5070 Ti Laptop GPU is capable; the initial obstacle was automatic CUDA toolchain discovery, not insufficient hardware or a failed propagation idea.

ELI5: instead of saving one photograph of every satellite for every second, the file saves each satellite's recipe. The GPU rapidly cooks the requested positions from those recipes. That is why the stored file stays tiny.

19.35 M	91.974 ms	717 / 717	0
full 7-day candidate intervals	direct on-demand query, p50	pass events with exact identity	propagation and sanitizer errors

Decision

Question	Answer	Evidence
Does full SGP4/SDP4 execute?	YES	Near-Earth, deep non-resonant, synchronous, and half-day resonance branches all passed.
Is the laptop powerful enough?	YES	About 208-212 million interval evaluations/s; 70 C maximum in a 98 s sustained run.
Is the compact representation real?	YES, with wording caveat	5,793-byte seed container replaces a chosen 590.6 MiB dense position buffer.
Is it an operational navigation system?	NO	TEME-to-PEF uses GMST+DUT1 approximation; polar motion, full EOP, RF, clock, and integrity are absent.

Honest confidence statement: the bounded implementation claims are strongly validated. '100% correct for every possible input and mission' cannot be established by finite tests; that would require formal verification plus mission-specific certification.

ROOT CAUSE

What was actually lacking?

Three separate questions had been mixed together. The test evidence separates them cleanly.

Layer	Finding	Verdict
SGP4/SDP4 mathematics	All four propagation regimes execute and match reference results.	Not lacking
Laptop hardware	RTX 5070 Ti Laptop, 12,227 MiB VRAM, compute capability 12.0; native GPU code runs normally.	Not lacking
Build integration	The supplied Windows helper failed to discover CUDA 12.8 and then returned exit code 0 despite no executable/tests.	Lacking
Command-line workaround	Explicit CUDA compiler/toolset and KLB_CUDA_ARCH=120 produced a native sm_120 + PTX build.	Resolved for this run
Approach scope	Compact seeds + on-demand SGP4 is valid, but the ground-frame conversion is intentionally simplified.	Valid, bounded

Why the first attempt looked worse than it was

+ CMake's automatic Windows CUDA discovery did not select the installed CUDA 12.8 toolchain for the new compute-capability 12.0 GPU.
+ The helper script did not stop or propagate the external command failure, so it printed a misleading success path.
+ After explicit configuration, the Release executable was 2,991,104 bytes and contained both sgp4_bench.sm_120.cubin and sgp4_bench.sm_120.ptx.

Direct answer: it was primarily the package's build automation. Your hardware and the core SGP4 seedchain approach both worked once the correct CUDA target was supplied.

VERIFICATION DESIGN

How the result was challenged

The test campaign deliberately used several different kinds of oracle so that one shared implementation could not simply agree with itself.

Layer	Coverage	Result
Package integrity	130 manifest SHA-256 entries	130 valid; 0 missing or mismatched
Native build/test	MSVC + CUDA 12.8 + sm_120; CTest	Build passed; 3/3 tests passed
Published Vallado vectors	Near, deep non-resonant, half-day, synchronous, GPS-like	Maximum listed position difference 0.00682 mm
CPU vs GPU	256 GPS states + 40 all-branch states	0 error/method mismatches; sub-micrometre maximum
Independent codebase	Python sgp4 2.25 / WGS72; 328 states, including long offsets	0 errors; 0.265627 mm maximum position delta
Full 7-day oracle	19,353,600 candidate intervals	All aggregate counters exact; 0 propagation errors
Pass-event output	717 expected events	All event identities exact; timing inside tolerance
Adversarial runtime	memcheck, racecheck, synccheck, forced truncation	0 sanitizer findings; truncation correctly rejected

Four regimes, not just GPS

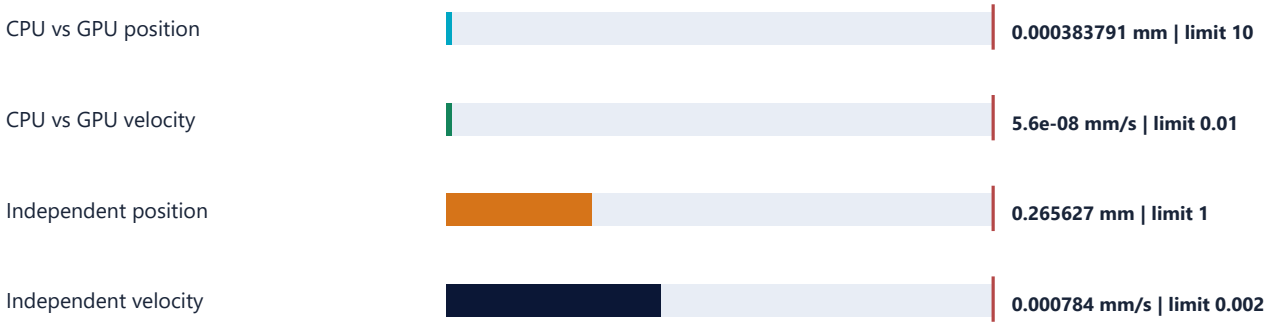
1	2	1	1
near-Earth object	deep non-resonant objects	synchronous resonance object	half-day resonance object

The default GPS dataset contains only deep non-resonant objects. The separate Vallado branch container was therefore essential to exercise the remaining branches on the physical GPU.

NUMERICAL ACCURACY

Implementation error is far below the acceptance limits

These measurements answer whether the code computes the selected SGP4 model consistently. They do not promise that an old OMM/TLE will predict the real satellite equally well.



Colored bar = measured maximum. Red marker = acceptance limit.

Comparison	States	RMS	Maximum	Limit / margin
GPS CPU vs GPU position	256	0.000070649 mm	0.000383791 mm	10 mm / 26,056x
GPS CPU vs GPU velocity	256	0.000000010 mm/s	0.000000056 mm/s	0.01 mm/s / 178,571x
All branches CPU vs GPU position	40	0.000072286 mm	0.000378879 mm	10 mm / 26,394x
Independent WGS72 position	328	0.063645 mm	0.265627 mm	1 mm audit gate / 3.76x
Independent WGS72 velocity	328	0.000507 mm/s	0.000784 mm/s	0.002 mm/s gate / 2.55x

The largest independent difference occurred for a reference object propagated roughly six years away from its epoch, an intentionally harsh numerical comparison. It still remained below 0.3 mm between implementations; physical orbit prediction at that age is a different question and would be much worse.

SEVEN-DAY RESULT

Pass scheduling matched end to end

The main workload used 32 GPS objects, a seven-day horizon, one-second sampling, and a ground station at 52 N, 5 E.

19,353,600	19,207,536	5,498,429	0
candidate intervals	supported and compatible	visible endpoints	propagation errors
359	**358**	**717**	**717**
acquisition-of-signal events	loss-of-signal events	total compacted events	exact event identities matched

Comparison	Identity	Max guard delta	Max crossing-time delta
GPU direct vs packaged CPU events	717 / 717 exact	8.372911e-9	0.000090514 s
GPU direct vs GPU dense	717 / 717 exact	2.775558e-16	5.820766e-11 s

Determinism nuance

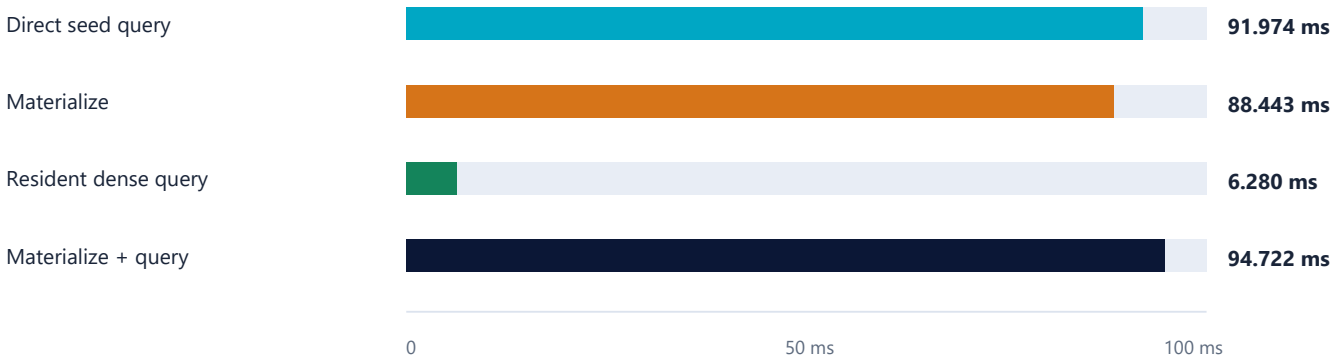
+ Repacking the KSGP1 container produced a byte-identical file with the same SHA-256.
+ Regenerating the event CSV produced every field, row, identity, and value identically.
+ Its file hash differed only because Windows emitted CRLF line endings while the package used LF. Therefore semantic reproduction is exact, but the documentation's cross-platform byte-for-byte CSV claim needs canonical newlines.

No silent data loss: forcing an event capacity of 1 for 11 produced events terminated with an explicit truncation error.

PERFORMANCE

One direct query is faster; repeated cached queries are different

For the stated seven-day workload, on-demand propagation avoids the cost of materializing a dense buffer and is about 3% faster end to end.



Path	p50	p95	p99	Meaning
Direct seed query	91.974 ms	92.000 ms	92.005 ms	Propagate and query in one pass
Materialize full SGP4	88.443 ms	-	-	Build 590.6 MiB double4 position buffer
Resident dense query	6.280 ms	-	-	Query an already-built position buffer
Dense end to end	94.722 ms	-	-	Materialization plus one query

Decision rule: for one pass over a horizon, direct seed propagation wins. If the exact same horizon will be queried two or more times, paying the materialization cost once and reusing the dense buffer can win on speed - if roughly 591 MiB of VRAM is acceptable.

210.4 M/s	207.7 M/s	212.2 M/s	14.65x
7-day direct throughput	1 GiB stress throughput	2 GiB stress throughput	resident dense query vs direct

The nearly constant throughput at 19.4M, 33.6M, and 67.1M candidates shows approximately linear scaling. Stress horizons extend far beyond the declared source-data horizon and are performance tests only.

COMPRESSION

The giant ratio is real - and it is avoided materialization

Calling this ordinary compression would be misleading. The package stores orbital elements and recomputes samples. The ratio therefore depends on how many time samples and which fields you choose as the hypothetical dense baseline.

LOG SCALE: each step right is about 10x more bytes

Baseline	Bytes	KSGP1 ratio	Interpretation
KSGP1 container	5,793	1.0x	32 portable seeds + timeline/metadata/container overhead
Source OMM CSV	4,852	0.84x	KSGP1 is actually 1.194x larger than the textual source
7-day float4 positions	309,658,112	53,453.84x	Avoided 16-byte position sample for each candidate
7-day double4 positions	619,316,224	106,907.69x	Measured GPU dense position-only baseline
7-day position + velocity baseline	928,974,336	160,361.53x	Higher baseline if both outputs are stored
2 GiB stress dense buffer	2,147,483,648	370,703.20x	Performance stress case, not extra information in the seed

Accurate wording: '5.8 KiB of seeds avoids materializing 590.6 MiB of seven-day double4 positions.' The ratio is horizon-dependent and compute is traded for storage.

GPU INTERNALS

Compute-bound, register-limited, not memory-bound

Nsight Compute and cuobjdump independently describe the same bottleneck: full SGP4 does substantial double-precision math per worker and uses many registers.

Kernel	Registers	Stack	Achieved occupancy	SM / FP64 throughput	DRAM
Direct seed query	146	48 B	23.72%	84.40% / 86.22%	0.008%
Materialize	138	48 B	23.74%	85.12% / 86.26%	0.721%
Dense query	40	0 B	95.82%	85.44% / 85.49%	10.23%
State validation	130	48 B	not profiled	not profiled	not profiled

+ No register spills were reported by ptxas, even though the direct kernel uses 146 registers.
+ Theoretical direct occupancy is 25%; achieved occupancy was 23.72%. This is the clearest optimization target.
+ Short-scoreboard and wait stalls dominate; DRAM traffic is negligible for direct propagation. More memory bandwidth is unlikely to fix the main bottleneck.
+ A focused event-writing launch recorded exactly 717 global atomic requests for 717 events. L2 atomic activity was only 0.00758%, so event compaction contention is negligible here.

Practical optimization priority: reduce live state/register pressure or split carefully chosen phases only if profiling proves the extra traffic is worthwhile. Replacing the laptop GPU is not the first recommendation.

RELIABILITY AND THERMALS

The laptop sustained the workload without instability

A 600-repeat direct-only run held the GPU busy for about 98.4 seconds. This is enough to expose immediate thermal throttling, though it is not an hours-long endurance qualification.

70 C	73.1 W	74.8 W	2,763 MHz
maximum active temperature	average active GPU power	maximum sustained-run power	average active graphics clock

Signal	Beginning	End / maximum	Interpretation
Temperature	55 C active start	70 C max; 69.9 C final-quarter average	Warm but stable in this run
Clock	2,744 MHz first-quarter average	2,767 MHz final-quarter average	No sustained clock collapse
Timing	161.55 ms p50 short run	162.33 ms p50 sustained	About 0.48% slower when thermally settled
VRAM	seed/direct workload	2,752 MiB maximum observed	Well inside 12,227 MiB

CUDA runtime safety

Tool	Scope	Result
Compute Sanitizer memcheck	All-branch validation + GPU smoke workload	0 errors
Compute Sanitizer racecheck	Event-writing direct smoke workload	0 hazards, 0 warnings
Compute Sanitizer synccheck	Direct smoke workload	0 errors
Negative truncation test	11 events into capacity 1	Explicit failure; no silent truncation

FEASIBILITY

What this can be used for now

The strongest fit is high-volume analytical screening where compact distribution, deterministic regeneration, and one-pass filtering matter more than maintaining a reusable dense ephemeris cache.

Use	Fit	Why
Constellation visibility screening	Strong	Millions of object-time candidates can be tested in under a tenth of a second for the demonstrated 32-object week.
Ground-station pass scheduling	Strong with frame caveat	717 pass boundaries reproduced with exact identities and sub-0.1 ms crossing-time delta.
Interactive scenario filtering	Strong	Seeds stay small; GPU recomputes only the selected horizon/criteria.
Large catalog batch analytics	Promising, not yet measured	Throughput scales linearly here, but thousands of objects and diverse regimes require a new benchmark.
Distribution or archival of dense trajectories	Strong alternative	A 5.8 KiB recipe can replace a chosen hundreds-of-MiB derived buffer.
Repeated queries over one frozen horizon	Conditional	Dense caching becomes faster after approximately the second query if VRAM is available.
Operational navigation / collision avoidance	Not ready	Requires current elements, frame/EOP rigor, covariance/uncertainty, validation, and mission certification.

Best current product description: a GPU-native full-SGP4 analytical query substrate with a portable seed container - not a compressed truth ephemeris and not a complete navigation stack.

LIMITS AND FIXES

What must be improved before stronger claims

None of these findings invalidates the core compute idea. They define the boundary between a successful technical substrate and a production-grade orbital service.

Priority	Finding	Recommended action
P1	Windows build helper hides failed CMake/build/test commands.	Check and propagate every external exit code; verify expected binaries exist before reporting success.
P1	Operational Earth-fixed accuracy is not in scope.	Add UT1/EOP ingestion, polar motion, TEME-to-ITRF validation, station altitude/mask, and explicit time standards.
P1	Model accuracy depends on fresh orbital elements.	Define element-age limits and compare predictions against authoritative ephemerides over mission-relevant horizons.
P2	Cross-platform event CSV hash is not reproducible because newline conventions differ.	Write canonical LF or document semantic rather than byte identity.
P2	Default GPS validation exercises only deep non-resonant objects.	Make the all-branch Vallado container part of the standard GPU acceptance command.
P2	Direct kernel is register-bound at about 24% occupancy.	Profile register-lifetime reductions; preserve correctness and measure end-to-end impact.
P3	Thermal run lasted 98 seconds, not hours.	Add 30-60 minute endurance and repeatability runs if continuous service is intended.

Go / no-go: GO for further engineering, demonstrations, and bounded analytical workloads. NO-GO for claims of operational navigation accuracy or universal correctness until the scope items above are implemented and independently certified.

EVIDENCE INDEX

Reproducible artifacts from this verification

All paths below are relative to the package root. Console logs preserve the actual outputs; CSV and Nsight reports preserve raw metrics. The report itself is a summary, not a substitute for those artifacts.

Evidence group	Files
Integrity / build	verification_build_windows_console.txt; verification_configure_cuda128_explicit_console.txt; verification_build_cuda128_console.txt; verification_ctest_console.txt
Binary architecture	verification_sgp4_cuobjdump_elf.txt; verification_sgp4_cuobjdump_ptx.txt; verification_sgp4_cuobjdump_resources.txt
Reference accuracy	verification_sgp4_reference_tests_console.txt; verification_gpu_gps_validation_console.txt; verification_gpu_all_branches_validation_console.txt
Independent comparison	verification_independent_sgp4_summary.json; verification_independent_sgp4_details.csv; verification_independent_sgp4_console.txt
Full workload	verification_sgp4_file_console.txt; sgp4_file_results.csv; verification_sgp4_full_cpu_oracle_console.txt
Stress / thermals	verification_sgp4_laptop_console.txt; verification_sgp4_2gib_console.txt; verification_sgp4_sustained_direct_console.txt; sgp4_sustained_direct_gpu_telemetry.csv
Sanitizers / negative	verification_compute_sanitizer_branches_console.txt; verification_compute_sanitizer_smoke_console.txt; verification_compute_sanitizer_racecheck_console.txt; verification_compute_sanitizer_synccheck_console.txt; verification_event_truncation_negative_console.txt
Profiling	sgp4_file_ncu.ncu-rep; sgp4_file_ncu_raw.csv; sgp4_event_ncu.ncu-rep; sgp4_event_ncu_raw.csv; verification_profile_sgp4_file_console.txt
Determinism	verification_repack_console.txt; verification_repacked_gps.ksgp; verification_regenerated_pass_console.txt; verification_regenerated_pass_events.csv

System and method notes

+ GPU: NVIDIA GeForce RTX 5070 Ti Laptop GPU, compute capability 12.0, 12,227 MiB VRAM.
+ Toolchain: NVIDIA driver 591.59, CUDA 12.8.61, CMake 4.3.2, MSVC 19.44; Release target KLB_CUDA_ARCH=120.
+ Independent library: Python sgp4 2.25 in WGS72 mode. This is an independent codebase comparison but uses the same published SGP4 model lineage.
+ Primary local model attribution: Vallado/CSSI SGP4/SDP4 implementation and the package's NOTICE_SGP4.md, validation documentation, and five-vector branch set.

Final verdict: the full-SGP4 seedchain mechanism is technically feasible and strongly validated on this laptop. The remaining gaps are build robustness, wording precision, and operational-system scope - not a failure of SGP4 or insufficient laptop hardware.